

Automated Deployment and Rollback

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 21: CI/CD and AI Automation

1. The Nature of Deployment Risk

Every deployment is a risk event — not because developers are careless, but because production environments can never be perfectly replicated in testing. Traffic patterns, database volumes, cached state, third-party integrations, and subtle infrastructure differences all create conditions that staging approximates but does not duplicate. Security practitioners must understand this reality because it shapes how deployment processes are designed, monitored, and controlled.

The goal of modern deployment engineering is not to eliminate risk — it is to detect problems within seconds of their emergence and respond faster than most users can notice. This requires combining deployment strategies that limit blast radius, automated monitoring that detects anomalies before thresholds are breached, and rollback mechanisms that operate without waiting for human authorization.

From a security perspective, deployment is also a moment of heightened exposure. The pipeline is temporarily reconfiguring load balancers, launching new service versions, and altering network paths. Sophisticated attackers time their activity to exploit this transitional state. Understanding deployment risk means understanding both operational failure risk and adversarial exploitation risk simultaneously.

2. Deployment Strategies Compared

Three primary strategies govern how changes move from artifact to production. Each trades infrastructure cost, rollback speed, and operational complexity differently. Choosing the wrong strategy for a deployment's risk profile is itself a risk management failure.

Strategy	How It Works	Rollback Speed	Best For
Blue-Green	Deploy to idle environment; switch load balancer when validated. Previous environment stays live.	Seconds — flip load balancer back	High-risk, server-side deployments where instant rollback is non-negotiable
Canary	Route a small percentage (5-10%) to new version; expand gradually as metrics confirm health.	Seconds — route traffic back; only small % affected	Any change where gradual exposure limits blast radius before full rollout
Rolling	Update instances batch by batch (e.g., 10% at a time); monitor between batches.	Moderate — already-updated instances must be reverted individually	Changes with moderate risk; default strategy in Kubernetes environments

Blue-green requires approximately double the infrastructure during deployment windows but provides the fastest and cleanest rollback. Canary provides finer-grained blast radius control and is often combined with AI-driven metric comparison between the canary population and the control population.

Rolling deployment is operationally simpler but complicates rollback because some instances are already updated when problems emerge.

3. Real-World Incident: CrowdStrike Falcon Sensor Update (2024)

Real-World Incident — CrowdStrike (July 2024): CrowdStrike deployed a content configuration update to its Falcon sensor software that caused approximately 8.5 million Windows systems to crash with a Blue Screen of Death (BSOD). The update passed internal testing but contained a logic error triggered only under specific production conditions. Airlines grounded flights, hospitals reverted to manual processes, and financial institutions suspended trading. Recovery required manual intervention on every affected endpoint because systems could not boot to receive a remediation update.

This incident demonstrates three critical lessons for SecAI+ candidates. First, even security-focused organizations with mature engineering practices deploy breaking changes — zero-risk deployment does not exist. Second, rollback mechanisms that depend on the affected system being able to boot are insufficient for low-level software components like kernel drivers or endpoint sensors. Third, the blast radius was total rather than gradual because CrowdStrike's delivery mechanism pushed to all endpoints simultaneously, without a canary phase that would have caught the failure before it reached millions of systems.

The counterfactual is instructive: a canary rollout delivering the update to 1% of endpoints first, combined with automated monitoring for system stability metrics, could have detected the BSOD pattern within minutes and halted the rollout before it reached the remaining 99% of the fleet. For the exam, CrowdStrike is the canonical example of what happens when rollback automation is absent or insufficient at scale.

4. AI-Powered Pre-Deployment Risk Assessment

Before a deployment begins, AI-driven risk assessment evaluates multiple signals to classify the deployment's risk level and select an appropriate strategy. This is fundamentally different from static deployment checklists — AI synthesizes dynamic context from across the system to make a risk judgment specific to this deployment at this moment.

- **Scope of change:** A one-line configuration fix carries less risk than a refactor touching 40 files across three services. AI counts changed files, functions, and lines as a risk signal.
- **Historical deployment behavior:** Services with a recent history of deployment failures receive higher caution ratings. AI queries deployment logs to surface this pattern.
- **Current production health:** Deploying to a system already showing elevated error rates or elevated latency is riskier than deploying to a healthy baseline. AI checks live metrics before authorizing deployment to begin.
- **Timing and traffic patterns:** High-traffic periods amplify the blast radius of failures. AI compares the proposed deployment time against historical traffic curves and flags high-traffic windows.
- **Dependency chain impact:** Changes to shared libraries or platform services affect more consumers than changes to isolated microservices. AI maps the dependency graph to assess downstream exposure.

The risk assessment output is not advisory — it drives automated decisions. A low-risk classification might allow automated canary deployment to proceed without human approval gates. A high-risk classification might require a smaller initial canary percentage, slower expansion intervals, and mandatory manual approval before each expansion stage.

5. Automated Validation Gates

At each stage of a phased deployment, automated validation gates determine whether to proceed, pause, or roll back. These gates run without human intervention — the deployment either satisfies the criteria or it does not. Understanding the hierarchy of gate types is important for the exam.

- **Smoke tests:** Verify that the application starts and fundamental endpoints are reachable. These run first because there is no value in running deeper tests if the service cannot start.
- **Health checks:** Confirm that all service endpoints return correct HTTP status codes and that internal health check routes report healthy state across all instances.
- **Synthetic transaction tests:** Execute scripted user journeys — login, search, add to cart, complete purchase — to verify that application logic works end-to-end, not just that the service is responding.
- **Performance tests:** Confirm that response latency at p50, p95, and p99 percentiles and throughput in requests per second meet SLA thresholds under representative load.
- **Business metric checks:** Verify that conversion rates, transaction volumes, or other outcome metrics are within expected ranges. Business metric gates catch logical errors that pass all technical tests.
- **Security gates:** Scan the deployed artifact for known CVEs, confirm that TLS certificates are valid, verify that security headers are present in responses, and confirm that authentication endpoints reject unauthorized requests.

6. Rollback Mechanics and Automatic Trigger Logic

When monitoring detects a problem, rollback must begin automatically. Every minute of manual investigation before rollback authorization is additional users experiencing the degraded state. Automatic rollback is not a sign that deployment automation has failed — it is the deployment automation working exactly as designed.

The trigger logic for automatic rollback relies on anomaly detection rather than simple threshold crossing. An error rate that rises from 0.05% to 0.12% may never breach a 1% threshold alert, but it represents a 140% relative increase that is statistically anomalous and warrants immediate investigation. AI-powered anomaly detection identifies these relative deviations against a rolling baseline; simple threshold alerts miss them entirely.

Key Principle: Rollback itself requires monitoring. After rollback completes, verify that metrics return to the pre-deployment baseline. If metrics do not improve after rollback, the root cause is not the new deployment — it may be in an external dependency, a database migration that cannot be reversed, or an infrastructure change made separately from the application deployment.

For blue-green deployments, rollback is a load balancer configuration change — a matter of seconds. For canary deployments, it means routing all traffic back to the stable version and decommissioning canary

instances — also fast, and with limited blast radius because only the canary percentage was affected. For rolling deployments, already-updated instances must be individually reverted, which takes longer and requires tracking which instances have been updated.

7. Deployment as an Attack Vector

Red-team thinking applied to deployment pipelines reveals that the deployment process itself is an attack surface. Sophisticated adversaries do not always attack running software — they attack the pipeline that produces and delivers it, because a successful pipeline compromise delivers malicious code through trusted channels with all the authority of a legitimate deployment.

- Supply chain insertion: An attacker who compromises a build system or artifact repository can insert malicious code into a build that passes all tests, receives all approvals, and gets deployed to production through the normal process. SolarWinds is the canonical example.
- Insider threat via rollback: An authorized team member deploys a backdoored version, allows it to execute for a brief period, then claims a technical error and triggers rollback to remove the malicious version before forensic investigation can begin. Audit logs and artifact integrity verification are the defenses.
- Transitional state exploitation: During deployment, load balancers are reconfiguring, new service versions are starting, and health checks are completing. This transitional window can expose attack surfaces that do not exist in steady-state operation. Attackers who observe deployment patterns can time attacks to this window.
- Pipeline credential theft: CI/CD systems require credentials to deploy to production environments. These credentials are high-value targets. Compromising a CI/CD system's secret storage yields production deployment capability without requiring exploitation of the application itself.

8. Database Migration Management

Database schema changes are the hardest category of deployment change to automate safely, because many database operations cannot be reversed without data loss. Dropping a column permanently removes data. Splitting a table into two tables requires data transformation that must be undone manually. Adding a NOT NULL constraint on a column with existing rows fails unless a default is provided. Application code can be swapped instantaneously with blue-green deployment; database schemas cannot.

Migration Pattern	What It Does	Key Advantage
Expand-Contract	Add new schema elements while keeping old ones. Transition the application to use new elements. Remove old elements in a later deployment cycle.	Each step is independently reversible; rollback at any stage is safe
Backward-Compatible Migration	Ensure new schema works with both old and new application versions during the transition window.	Enables zero-downtime deployment when both app versions run simultaneously during rollout
Separate Migration Timing	Run schema migrations as a separate, independently verified step before	Isolates migration failures from application deployment

	deploying application code.	failures; each can be rolled back independently
Data Versioning	Maintain version columns or shadow tables during migration; update in batches with monitoring.	Limits the blast radius of a migration error to the records processed so far

9. Feature Flags: Decoupling Deployment from Release

Feature flags address a fundamental problem in continuous deployment: code is ready to deploy to production long before the team is ready to release the feature to all users. Without feature flags, teams must choose between holding code back (reducing deployment frequency and increasing integration risk) or releasing features before confidence is established. Feature flags solve this by shipping code in a disabled state and activating features via configuration.

The security and operational implications of feature flags are significant. A feature that exhibits anomalous behavior can be disabled via a flag change that takes effect in seconds — no redeployment, no rollback of unrelated code. Features can be enabled for internal users or beta testers only, creating a natural canary population that tests the feature under real conditions before broad release. AI-driven flag management can automatically expand a feature's audience as confidence metrics accumulate and automatically disable the feature when anomalous behavior is detected.

Feature flag platforms worth knowing for the exam: LaunchDarkly provides enterprise-grade flag management with targeting rules, experimentation support, and direct CI/CD integration. Unleash is the leading open-source alternative. Flagsmith provides a cloud and self-hosted option with built-in A/B testing. All three integrate with monitoring systems to support automated flag state changes based on metric signals.

10. DORA Metrics: Measuring Deployment Health

The DevOps Research and Assessment (DORA) program identified four metrics that distinguish high-performing software delivery organizations from low-performing ones. These metrics are diagnostic: they reveal where bottlenecks and weaknesses in the deployment process live, which makes them directly relevant to deployment security posture assessment.

DORA Metric	What It Measures	Elite Performer Benchmark	Security Implication
Deployment Frequency	How often code is deployed to production	Multiple times per day	Higher frequency means smaller change batches and easier isolation of failure cause
Lead Time for Changes	Time from code commit to production deployment	Less than one hour	Long lead times indicate manual gates that also delay security patches
Change Failure Rate	Percentage of deployments that cause service degradation or	Below 15%	High failure rate indicates inadequate validation gates or insufficient testing

	require rollback		
Mean Time to Restore (MTTR)	Time to recover from a deployment failure	Less than one hour	High MTTR indicates insufficient rollback automation or poor incident response

AI-powered deployment analytics track DORA metrics continuously and surface bottlenecks. A high change failure rate combined with a low MTTR indicates that rollback works but validation gates are insufficient — failures are getting to production and being caught post-deployment. A high change failure rate combined with a high MTTR indicates a compounding problem: not only are failures reaching production, but recovery is slow. Both patterns warrant different remediation strategies.

11. Responsible Automation and Human Oversight

Sophisticated deployment automation creates a risk of misplaced confidence: if everything is monitored, gated, and automatically recoverable, do human reviewers add value? The answer is yes, for reasons that automation cannot eliminate. Automated gates catch what they have been programmed to catch. Novel failure modes, subtle business logic errors, emergent behaviors from complex system interactions, and adversarial manipulation of monitoring systems can all produce clean automated gate results while causing significant harm.

The principle of meaningful human oversight — central to both AI governance frameworks and to responsible deployment practice — requires that for high-risk deployments, a human reviews monitoring data during the canary phase before expansion proceeds. For production-affecting changes during business-critical periods, a qualified engineer must be reachable and capable of intervening. Automation is the first line of defense, not the only line.

An analogy for this principle: commercial aviation uses extensive automation for flight control, navigation, and monitoring. But the most sophisticated autopilot systems still require human pilots who understand the context the automation cannot — unusual weather situations, passenger safety concerns, and system states the automation was not designed for. The human is not in the loop to perform routine tasks; the human is in the loop to handle what automation cannot handle alone. Deployment automation works the same way.

Key Terms Glossary

Term	Definition
Blue-Green Deployment	Deployment strategy maintaining two identical production environments; traffic switches between them, enabling instant rollback by switching the load balancer back.
Canary Deployment	Deployment strategy routing a small percentage of traffic to the new version first, monitoring metrics, then gradually expanding rollout if the canary performs well.
Rolling Deployment	Deployment strategy updating instances batch by batch; simpler than blue-green but makes rollback more complex for already-updated instances.

Feature Flag	A configuration mechanism that ships code to production in a disabled state, allowing features to be enabled or disabled independently of code deployment.
Automated Validation Gate	A check that runs automatically at each deployment stage to determine whether to proceed, pause, or roll back — without human intervention.
DORA Metrics	Four metrics (deployment frequency, lead time, change failure rate, MTTR) that characterize software delivery performance, defined by the DevOps Research and Assessment program.
Expand-Contract Migration	A database migration pattern that adds new schema elements while retaining old ones, transitions application code, then removes old elements in a later deployment — each step independently reversible.
Synthetic Transaction Test	An automated test that executes a scripted end-to-end user journey (login, browse, purchase) in production to verify application logic, not just service availability.
Mean Time to Restore (MTTR)	The average time from detection of a deployment failure to restoration of normal service; a key DORA metric indicating rollback and incident response effectiveness.
Infrastructure as Code (IaC)	The practice of defining infrastructure in versioned code files (Terraform, Pulumi, AWS CDK) so environments are provisioned consistently and reproducibly.
Change Failure Rate	The percentage of deployments that cause a service degradation or incident requiring rollback; a DORA metric indicating validation gate effectiveness.
Blast Radius	The scope of users, services, or systems affected when a deployment failure occurs; canary and feature flag strategies minimize blast radius by limiting initial exposure.

Quick Revision Checklist

- Blue-green deployment enables instant rollback by keeping the previous environment live and switching the load balancer — no re-deployment needed.
- Canary deployment limits blast radius by routing a small percentage to the new version first and expanding only when metrics confirm health.
- Rolling deployment updates instances batch by batch and is the Kubernetes default — rollback is slower because some instances are already updated.
- AI pre-deployment risk assessment evaluates change scope, service history, current production health, timing, and dependency impact before deployment begins.
- Automated validation gates run smoke tests, health checks, synthetic transactions, performance tests, business metrics, and security checks — no human decides whether to proceed.
- Automatic rollback should not wait for human authorization — every minute of delay is additional users experiencing degradation.
- Rollback itself must be monitored: if metrics do not return to baseline after rollback, the root cause is not the new deployment.

- The CrowdStrike July 2024 incident demonstrates why endpoint software updates require canary rollout — a total-fleet simultaneous push with no staged rollback capability caused 8.5 million BSOD crashes.
- Deployment is an attack surface: supply chain insertion, insider rollback abuse, transitional-state exploitation, and pipeline credential theft are specific adversarial scenarios.
- Expand-contract database migration keeps old and new schema elements simultaneously, making each migration step independently reversible.
- Feature flags decouple deployment (when code reaches production) from release (when the feature activates for users), enabling instant disable without redeployment.
- DORA metrics: deployment frequency, lead time for changes, change failure rate, and MTTR. Elite performers deploy multiple times per day with MTTR under one hour.
- High change failure rate plus high MTTR means inadequate validation gates AND insufficient rollback automation — two distinct problems requiring separate fixes.
- Human oversight remains essential for high-risk deployments because automation only catches what it was programmed to catch; novel failure modes and adversarial manipulation can evade automated gates.
- IaC tools (Terraform, Pulumi, AWS CDK) ensure environment consistency; AI-powered scanners like Checkov and Bridgecrew detect security misconfigurations before provisioning.

10 Exam-Based MCQs — Automated Deployment and Rollback

Test yourself after reading. These questions are written in real exam style — scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. *Scenario:* A financial services firm experienced a production outage last month when a deployment introduced a regression that was not caught until customers reported failed transactions two hours after release. The CISO has asked the DevOps and security teams to recommend changes to the deployment process to reduce the risk of recurrence. The firm currently deploys directly to all production instances simultaneously. What should the security team recommend FIRST?

- A) Switch immediately to rolling deployment to reduce infrastructure costs
- B) Implement canary deployment with automated metric monitoring before expanding the rollout
- C) Require a 72-hour manual observation window after every deployment
- D) Disable automatic rollback to prevent unauthorized production changes

Answer: B **Explanation:** B is correct because canary deployment limits blast radius by routing a small percentage of traffic to the new version first, and automated metric monitoring detects anomalies before full rollout — addressing both the risk of failure and the speed of detection. A is wrong because rolling deployment reduces infrastructure cost but complicates rollback, which is the wrong trade-off when the primary concern is a recent deployment-caused outage. C is wrong because a 72-hour manual window eliminates the speed advantage of CI/CD and introduces significant delays to security patching without providing the automated detection that prevents recurrence. D is wrong because automatic rollback is a core defense against deployment failures — disabling it increases, not decreases, risk.

Q2. Scenario: A security architecture review board is analyzing the July 2024 CrowdStrike incident in which a content configuration update caused approximately 8.5 million Windows systems to crash. The board is examining what deployment controls would have prevented the widespread impact. The update was delivered simultaneously to the full endpoint fleet without a phased rollout mechanism. Which deployment strategy would MOST effectively have prevented the widespread impact of the CrowdStrike July 2024 incident?

- A) Blue-green deployment with instant load balancer rollback
- B) Rolling deployment with 10% batch sizes
- C) Canary deployment with automated health monitoring that halts rollout on anomaly detection
- D) Infrastructure as Code with Terraform to ensure environment consistency

Answer: C Explanation: C is correct because a canary rollout would have deployed the faulty update to a small percentage of endpoints first. Automated monitoring detecting BSOD events or system instability in the canary group would have halted the rollout before it reached the remaining fleet. A is wrong because blue-green deployment works for server-side deployments where load balancers can redirect traffic, but CrowdStrike's update was delivered to endpoint software — not a server environment with a switchable load balancer. B is wrong because rolling deployment with 10% batches would have eventually reduced the affected systems if monitoring halted the rollout, but without automated halt logic, the rollout would continue batch by batch; the key missing element is the automated monitoring trigger, not the batch size. D is wrong because IaC ensures environment consistency in infrastructure provisioning but does not affect how a software update is delivered to endpoints.

Q3. Scenario: A SaaS company is deploying a new version of its payment processing service using canary deployment, with 5% of traffic routed to the new version. Automated monitoring detects that the p99 latency for the canary instances is 340ms versus 85ms for the stable instances — a statistically significant deviation. The deployment automation has flagged this as an anomaly. Which action should the deployment system take AUTOMATICALLY when anomaly detection identifies the issue?

- A) Page the on-call engineer and wait for authorization before taking any action
- B) Immediately roll back all production instances to the previous version
- C) Halt further canary expansion and route all new traffic back to the stable version
- D) Increase the canary percentage to gather more data before deciding

Answer: C Explanation: C is correct because when an anomaly is detected during a canary deployment, the immediate automated response is to halt expansion and route traffic back to the stable version — the canary group was small precisely so that only a small percentage of users was affected, and the automation should exploit that containment immediately. A is wrong because waiting for human authorization before acting allows the problem to persist and potentially expand; automatic rollback is designed to eliminate this delay. B is wrong because rolling back all production instances is the correct response for a full-deployment failure, not a canary deployment — in a canary scenario, only canary instances need to be rolled back; the rest of production was never updated. D is wrong because increasing canary percentage when anomalies are detected expands the blast radius in the wrong direction.

Q4. Which database migration approach BEST enables rollback safety when adding a new column that will eventually replace an existing column?

- A) Drop the old column and add the new column in a single migration
- B) Use expand-contract migration: add the new column first, transition the application, then drop the old column in a later deployment
- C) Add the new column with a foreign key reference to the old column to maintain data integrity
- D) Use a rolling deployment to update the schema incrementally across database shards

Answer: B **Explanation:** B is correct because expand-contract migration makes each step independently reversible: adding the new column is non-destructive and can be rolled back; transitioning application code is independently deployable and rollbackable; dropping the old column only happens after confidence is established and is a separate, isolated change. A is wrong because dropping the old column in the same migration as adding the new column is irreversible — if the migration or application fails, the old column's data is gone. C is wrong because foreign key references between old and new columns within the same table is not a recognized migration pattern and would create redundant data storage without migration safety. D is wrong because rolling deployment is an application deployment strategy, not a database schema migration strategy; schema migrations require their own approach.

Q5. Which DORA metric most directly indicates that a team's rollback automation is insufficient?

- A) High deployment frequency
- B) Short lead time for changes
- C) High mean time to restore (MTTR)
- D) Low change failure rate

Answer: C **Explanation:** C is correct because MTTR measures how quickly a team recovers from a deployment failure. A high MTTR indicates that when failures occur, recovery is slow — which directly implies that rollback automation is either absent, slow, or requires extensive manual intervention. A is wrong because high deployment frequency is a positive indicator of delivery performance and says nothing about rollback capability. B is wrong because short lead time for changes indicates efficient CI/CD pipeline automation but does not speak to recovery speed when failures occur. D is wrong because a low change failure rate indicates that validation gates are effective at preventing bad deployments — this is good, but it does not indicate rollback capability.

Q6. What is the PRIMARY security advantage of feature flags over traditional deployment rollback?

- A) Feature flags encrypt feature code until activation, preventing unauthorized access
- B) Feature flags allow a feature to be disabled instantly via configuration without redeploying code
- C) Feature flags prevent unauthorized users from accessing the CI/CD pipeline
- D) Feature flags enforce code signing on all deployed artifacts

Answer: B **Explanation:** B is correct because the primary operational and security advantage of feature flags is the ability to disable a problematic feature in seconds via a configuration change — no code redeployment, no rollback of unrelated changes, and no disruption to other features that shipped in the same deployment. A is wrong because feature flags do not encrypt code; they control whether code paths

are executed at runtime. C is wrong because feature flags have no relationship to CI/CD pipeline access control; they operate at the application runtime level, not at the pipeline authorization level. D is wrong because code signing is a separate security control applied to artifacts during the build process, unrelated to feature flag functionality.

Q7. Which automated validation gate type would MOST likely catch a regression in application business logic that does not affect service availability or response latency?

- A) Health check
- B) Smoke test
- C) Synthetic transaction test
- D) Performance test

Answer: C **Explanation:** C is correct because synthetic transaction tests execute scripted end-to-end user journeys — login, browse, add to cart, complete purchase — and verify that application logic produces the correct outcomes. A business logic regression (e.g., incorrect discount calculation, incorrect order status update) would cause a synthetic transaction to produce wrong results or fail, catching the regression. A is wrong because health checks verify that service endpoints return correct HTTP status codes and report healthy state — they confirm the service is running, not that it is computing correct results. B is wrong because smoke tests verify that the application starts and basic functionality responds — also a liveness check, not a logic verification. D is wrong because performance tests verify response latency and throughput against SLAs — they measure speed, not correctness of business logic.

Q8. An attacker who has compromised an authorized developer's CI/CD credentials wants to insert malicious code into production with minimal forensic trace. Which technique BEST achieves this goal?

- A) Submit a pull request with malicious code and wait for approval
- B) Deploy a backdoored version to production, allow it to execute briefly, then trigger rollback and claim a technical error
- C) Modify the production database directly using stolen database credentials
- D) Launch a DDoS attack against the production environment during the next deployment window

Answer: B **Explanation:** B is correct because this technique exploits rollback as a forensic cover: the malicious version executes, performs its intended action (data exfiltration, credential harvest, etc.), and is then removed from production via rollback triggered by the attacker themselves. If logs are not preserved for the rolled-back version, the malicious code may never be forensically examined. The defense is immutable audit logs of all deployment artifacts and deployed versions, including rolled-back ones. A is wrong because a pull request requires code review approval — it introduces human review that would likely detect malicious changes, and it leaves a permanent record in version control. C is wrong because direct database modification, while serious, does not use the CI/CD vector described in the question and leaves database audit logs as evidence. D is wrong because a DDoS attack is a disruption tactic, not a covert code insertion technique.

Q9. Which IaC security scanning tool is designed to detect misconfigurations in Terraform and CloudFormation templates before infrastructure is provisioned?

- A) SonarQube
- B) Checkov
- C) Snyk
- D) Dependabot

Answer: B **Explanation:** B is correct because Checkov (developed by Bridgecrew, now part of Palo Alto Networks) is a static analysis tool specifically designed for Infrastructure as Code security scanning. It analyzes Terraform, CloudFormation, Kubernetes YAML, ARM templates, and other IaC formats for security misconfigurations such as publicly accessible S3 buckets, overly permissive security groups, and disabled encryption. A is wrong because SonarQube is a code quality and security analysis platform for application source code (Java, Python, JavaScript, etc.), not for IaC template analysis. C is wrong because Snyk is primarily a software composition analysis tool for detecting vulnerabilities in application dependencies; while Snyk does have some IaC scanning capability, Checkov is the purpose-built IaC security tool specified in the course. D is wrong because Dependabot is a GitHub tool that automatically creates pull requests to update outdated dependencies — it is a dependency management tool, not an IaC security scanner.

Q10. A team is observing a change failure rate of 32% and an MTTR of 4 hours. Which combination of improvements would MOST directly address both metrics?

- A) Increase deployment frequency and reduce lead time for changes
- B) Strengthen automated validation gates and implement automatic rollback
- C) Implement Infrastructure as Code and standardize development environments
- D) Add more manual approval stages to prevent bad deployments from reaching production

Answer: B **Explanation:** B is correct because a high change failure rate indicates that deployments are reaching production with defects — strengthening automated validation gates catches more defects before they reach production, reducing the failure rate. A high MTTR indicates that recovery from failures is slow — implementing automatic rollback eliminates the manual investigation and authorization delay, reducing MTTR to seconds. The two improvements directly target the two metrics. A is wrong because deployment frequency and lead time are DORA metrics measuring delivery speed, not deployment quality or recovery speed; improving them does not address why deployments fail or why recovery takes 4 hours. C is wrong because IaC and environment standardization reduce environment-inconsistency failures, which is a subset of failure causes; they do not address the full range of reasons for a 32% failure rate or the MTTR problem. D is wrong because additional manual approval stages may reduce deployment frequency without improving validation quality — human reviewers conducting manual reviews introduce delay without the systematic coverage that automated tests provide, and they do not address MTTR at all.